

DESIGN PATTERN

ETAT

SOMMAIRE

1. Introduction

5. Role des classes

9. Live Coding

2. Pattern du GoF

6. Lien avec SOLID

10. QCM

3. Exemple Pizza

7. Limites du pattern

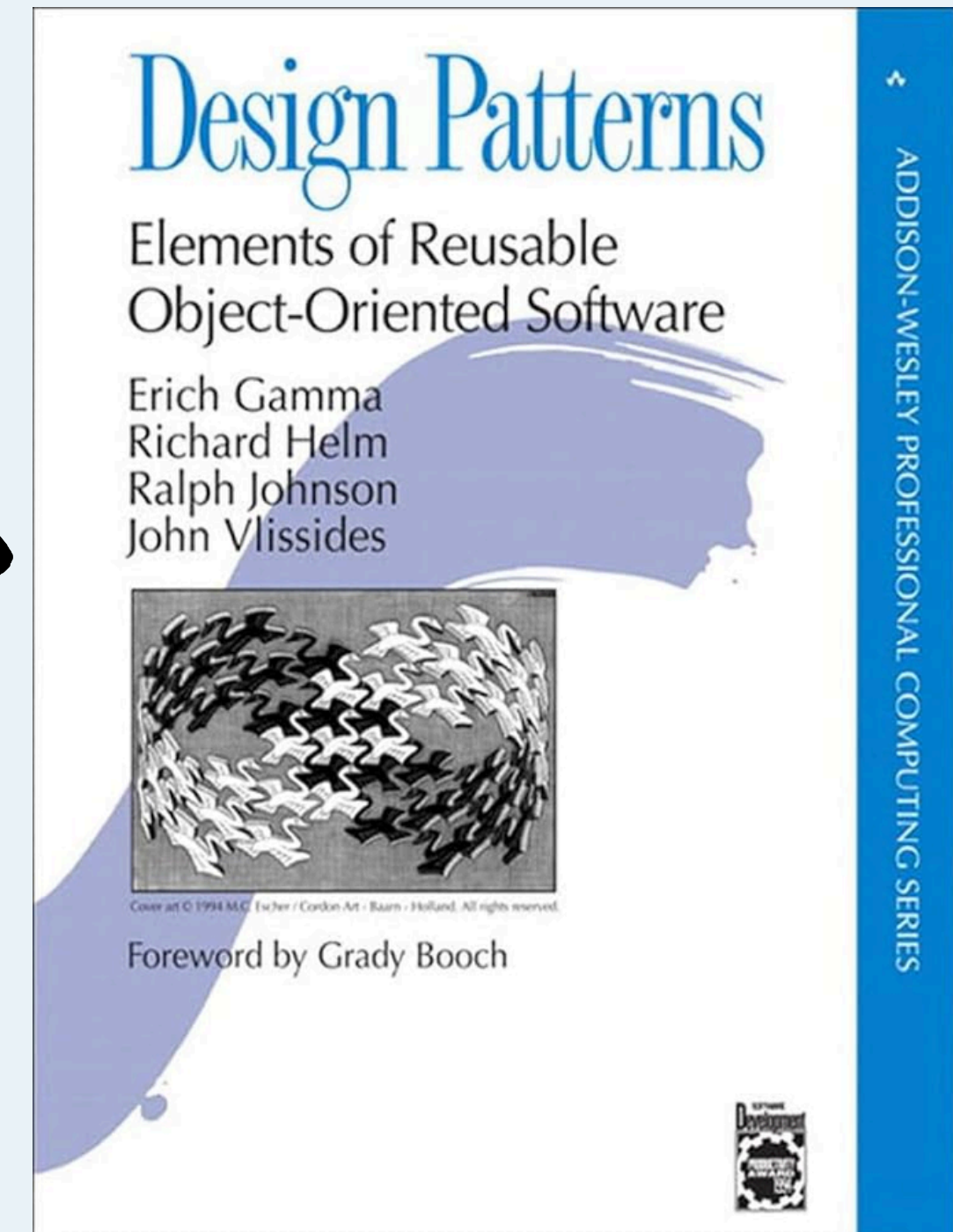
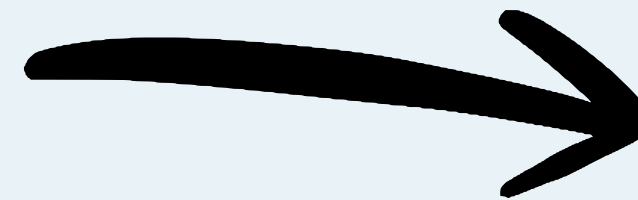
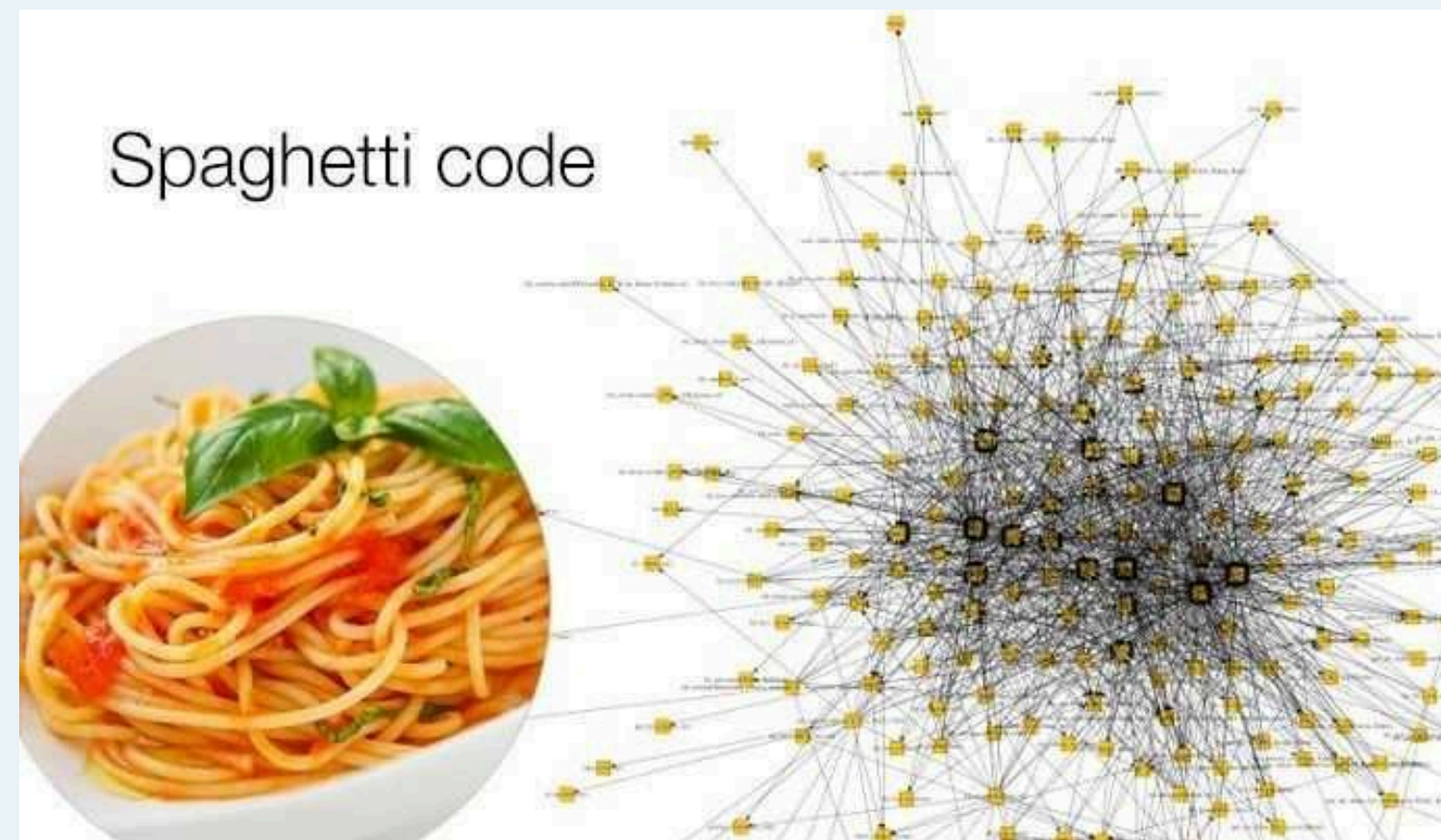
11. Sitographie

4. Solution par la littérature

8. Lien avec STRATEGIE

12. IAGraphie

INTRODUCTION



PATTERN DU GoF

THE 23 GANG OF FOUR DESIGN PATTERNS

C Abstract Factory	S Facade	S Proxy
S Adapter	C Factory Method	B Observer
S Bridge	S Flyweight	C Singleton
C Builder	B Interpreter	B State
B Chain of Responsibility	B Iterator	B Strategy
B Command	B Mediator	B Template Method
S Composite	B Memento	B Visitor
S Decorator	C Prototype	

3 Familles

Comportementaux : B

Structurels : S

Créatifs : C

EXAMPLE

Pizza Nizar



BAD CODE

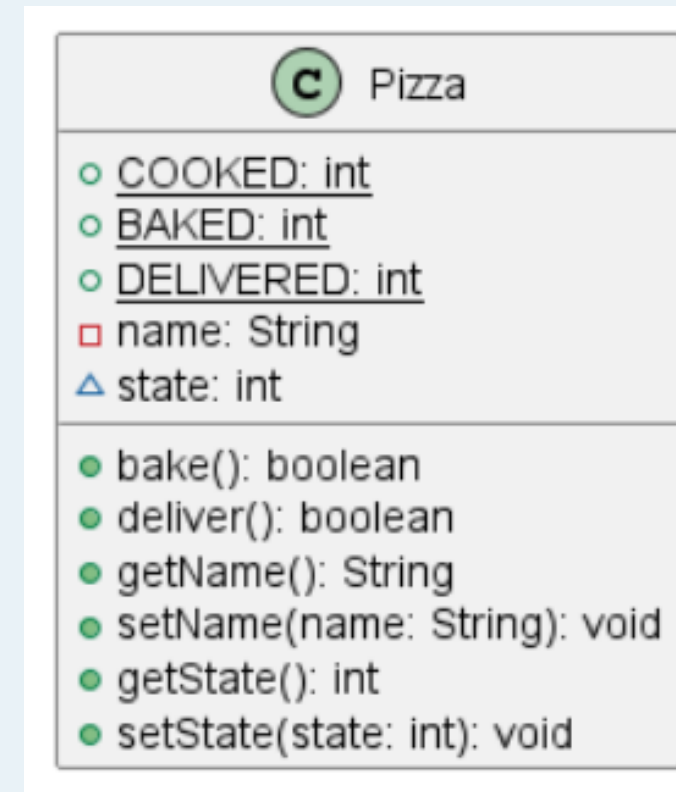
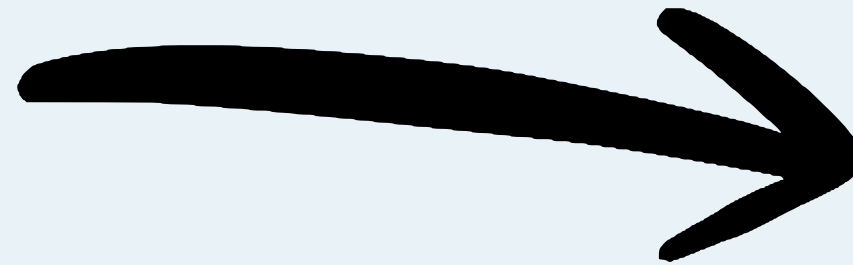
Ouuhh
le méchant
code Smell

```
public class Pizza {
    public static final int COOKED = 0;
    public static final int BAKED = 1;
    public static final int DELIVERED = 2;
    private String name;
    int state = COOKED;
    public boolean bake() {
        if (state == COOKED) {
            state = BAKED;
        } else if (state == BAKED) {
            return false; // Can't bake a pizza already baked !
        } else if (state == DELIVERED) {
            return false; // Can't bake a pizza already delivered !
        }
        return true;
    }
    public boolean deliver() {
        if (state == COOKED) {
            return false; // Can't deliver a pizza not baked yet;
        } else if (state == BAKED) {
            state = DELIVERED;
        } else if (state == DELIVERED) {
            return false; // Can't deliver a pizza already delivered;
        }
        return true;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public int getState() {
        return state;
    }
    public void setState(int state) {
        this.state = state;
    }
}
```

C Pizza	
●	<u>COOKED</u> : int
●	<u>BAKED</u> : int
●	<u>DELIVERED</u> : int
□	name: String
△	state: int
●	bake(): boolean
●	deliver(): boolean
●	getName(): String
●	setName(name: String): void
●	getState(): int
●	setState(state: int): void

QUELS SONT DONC LES PROBLÈMES ?

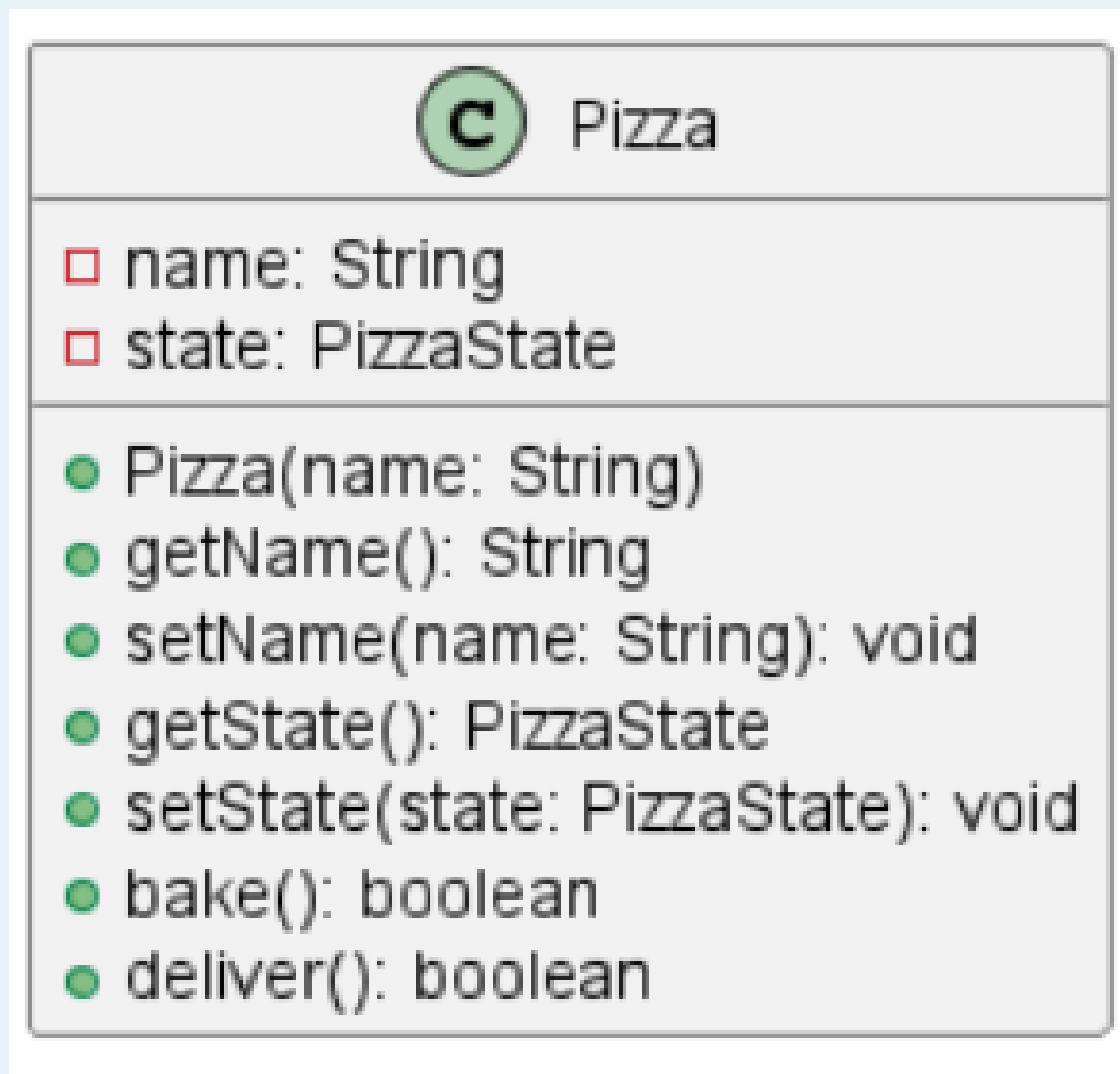
- Viol de SRP
- Viol de OCP
- Complexité cyclomatique
 - Cascade de if / else if / else



```
public boolean bake() {
    if (state == COOKED) {
        state = BAKED;
    } else if (state == BAKED) {
        return false; // Can't bake a pizza already baked !
    } else if (state == DELIVERED) {
        return false; // Can't bake a pizza already delivered !
    }
    return true;
}
```

PROPOSITION DE CORRECTION

CLASSE PIZZA



```
public class Pizza {
    private String name;
    private PizzaState state;

    public Pizza(String name) {
        this.name = name;
        this.state = new CookedState(); // État initial par défaut
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public PizzaState getState() {
        return state;
    }

    public void setState(PizzaState state) {
        this.state = state;
    }

    public boolean bake() {
        return state.bake(this);
    }

    public boolean deliver() {
        return state.deliver(this);
    }
}
```

INTERFACE PizzaState

```
public interface PizzaState {  
    boolean bake(Pizza pizza);  
    boolean deliver(Pizza pizza);  
}
```

I <i>PizzaState</i>
bake(pizza: Pizza): boolean deliver(pizza: Pizza): boolean

CLASSE CookedState

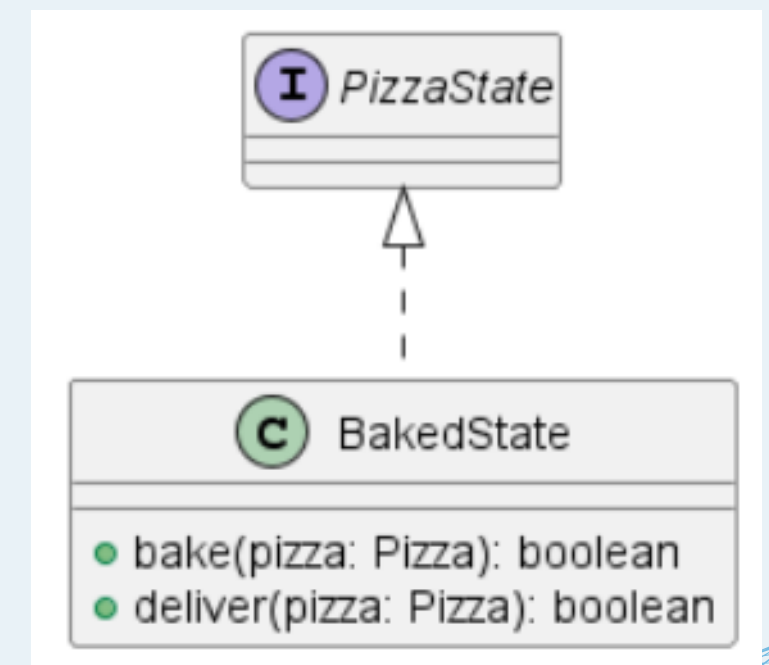
```
public class CookedState implements PizzaState {  
  
    @Override  
    public boolean bake(Pizza pizza) {  
        System.out.println("Cuisson de la pizza en cours...");  
        pizza.setState(new BakedState());  
        return true;  
    }  
  
    @Override  
    public boolean deliver(Pizza pizza) {  
        System.out.println("Erreur : Impossible de livrer une pizza pas encore cuite !");  
        return false;  
    }  
}
```

I *PizzaState*

bake(pizza: Pizza): boolean
deliver(pizza: Pizza): boolean

CLASSE BakedState

```
public class BakedState implements PizzaState {  
  
    @Override  
    public boolean bake(Pizza pizza) {  
        System.out.println("Erreur : Impossible de recuire une pizza déjà cuite !");  
        return false;  
    }  
  
    @Override  
    public boolean deliver(Pizza pizza) {  
        System.out.println("Livraison de la pizza en cours...");  
        pizza.setState(new DeliveredState());  
        return true;  
    }  
}
```



CLASSE DeliveredState

```
public class DeliveredState implements PizzaState {  
  
    @Override  
    public boolean bake(Pizza pizza) {  
        System.out.println("Erreur : Impossible de cuire une pizza déjà livrée !");  
        return false;  
    }  
  
    @Override  
    public boolean deliver(Pizza pizza) {  
        System.out.println("Erreur : Impossible de livrer une pizza déjà livrée !");  
        return false;  
    }  
}
```

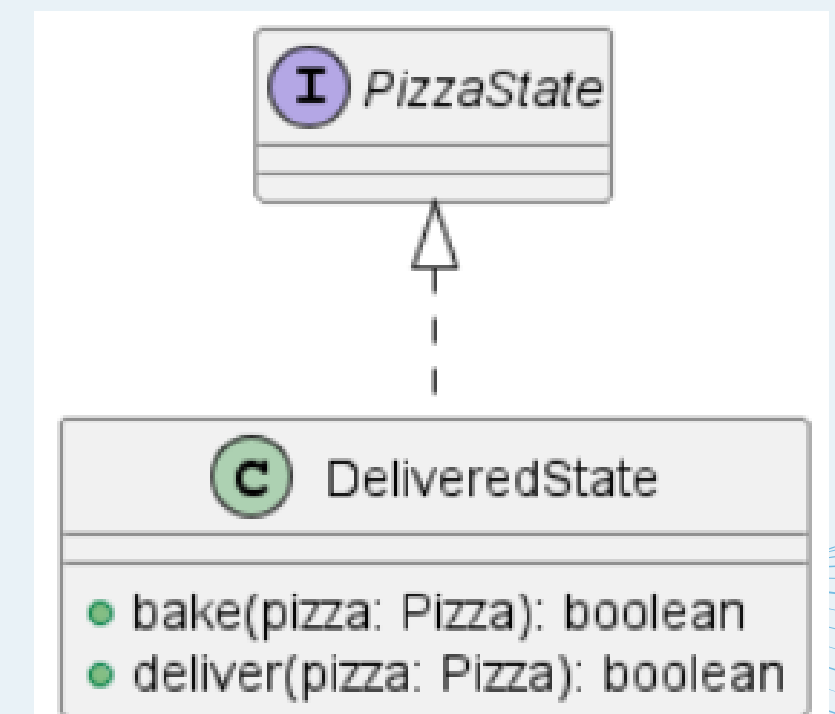


DIAGRAMME DE CLASSE

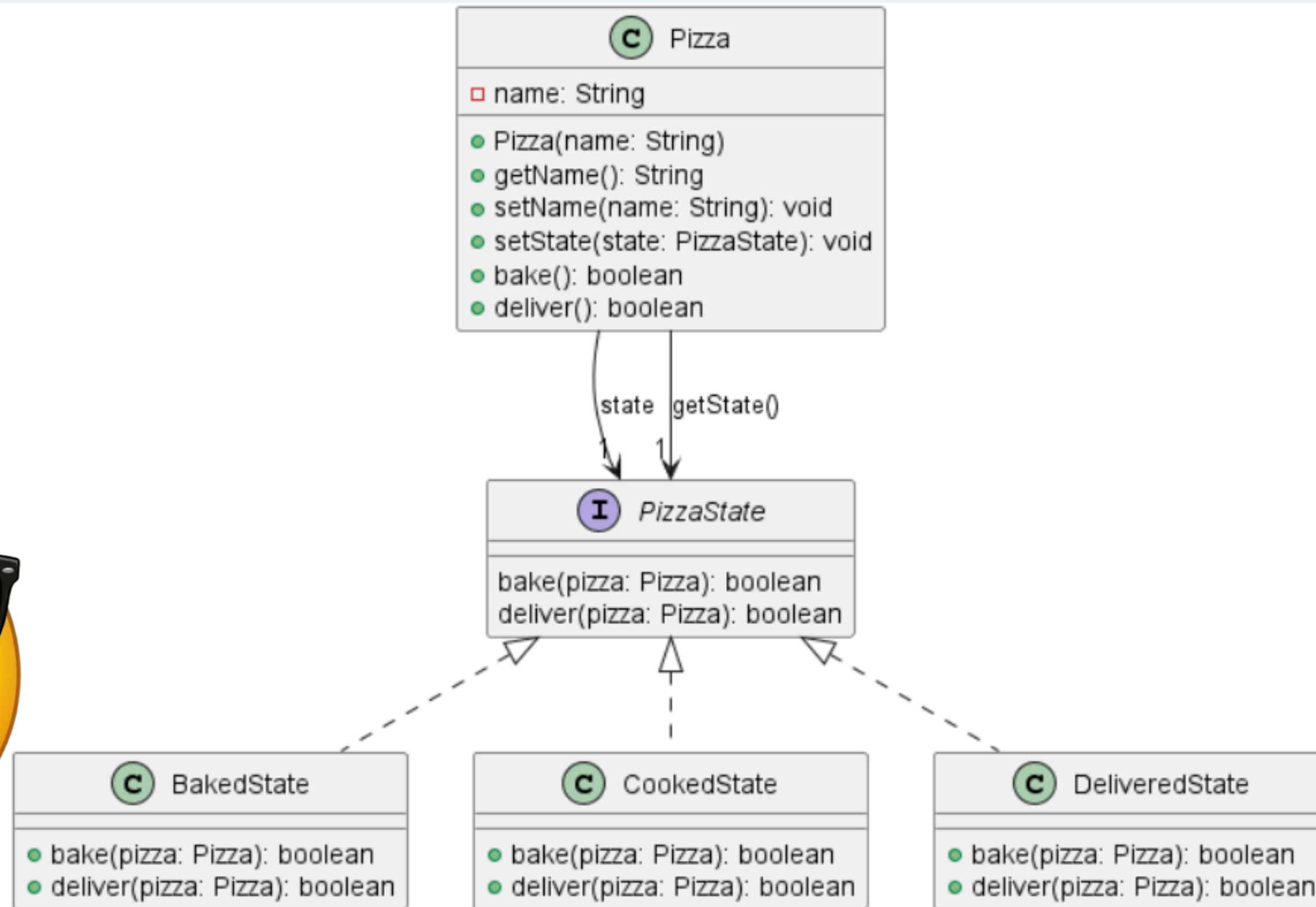
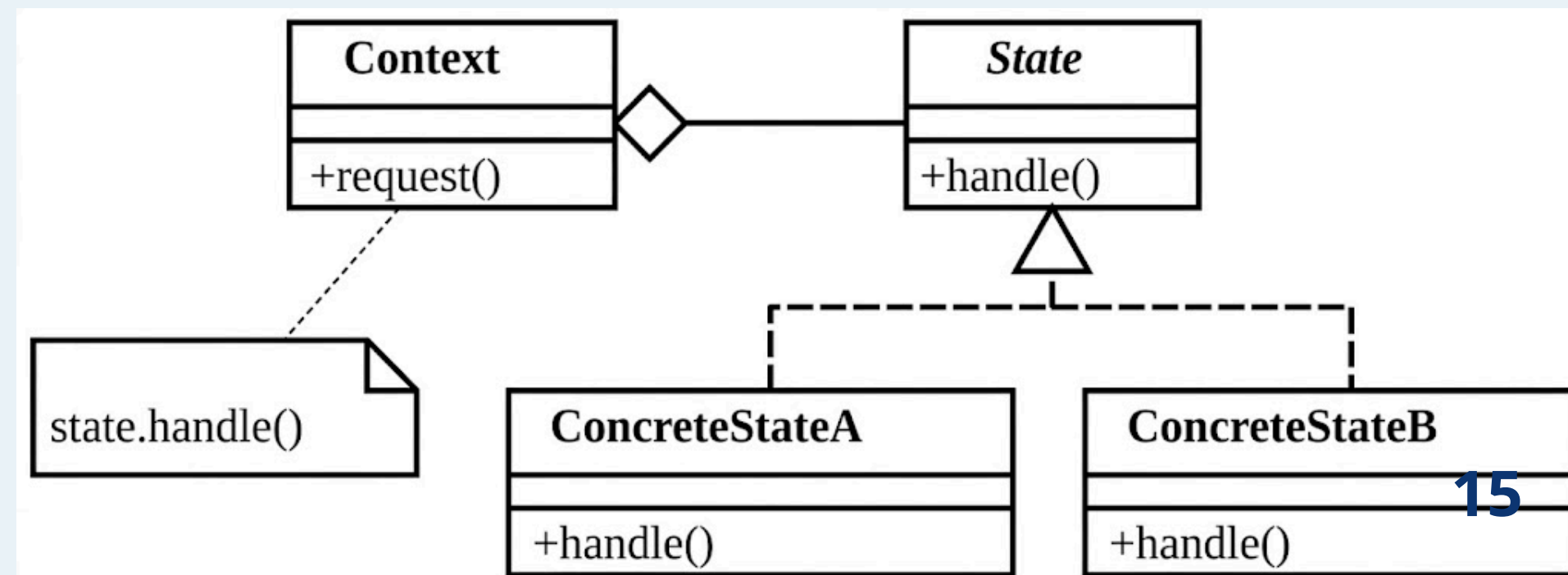
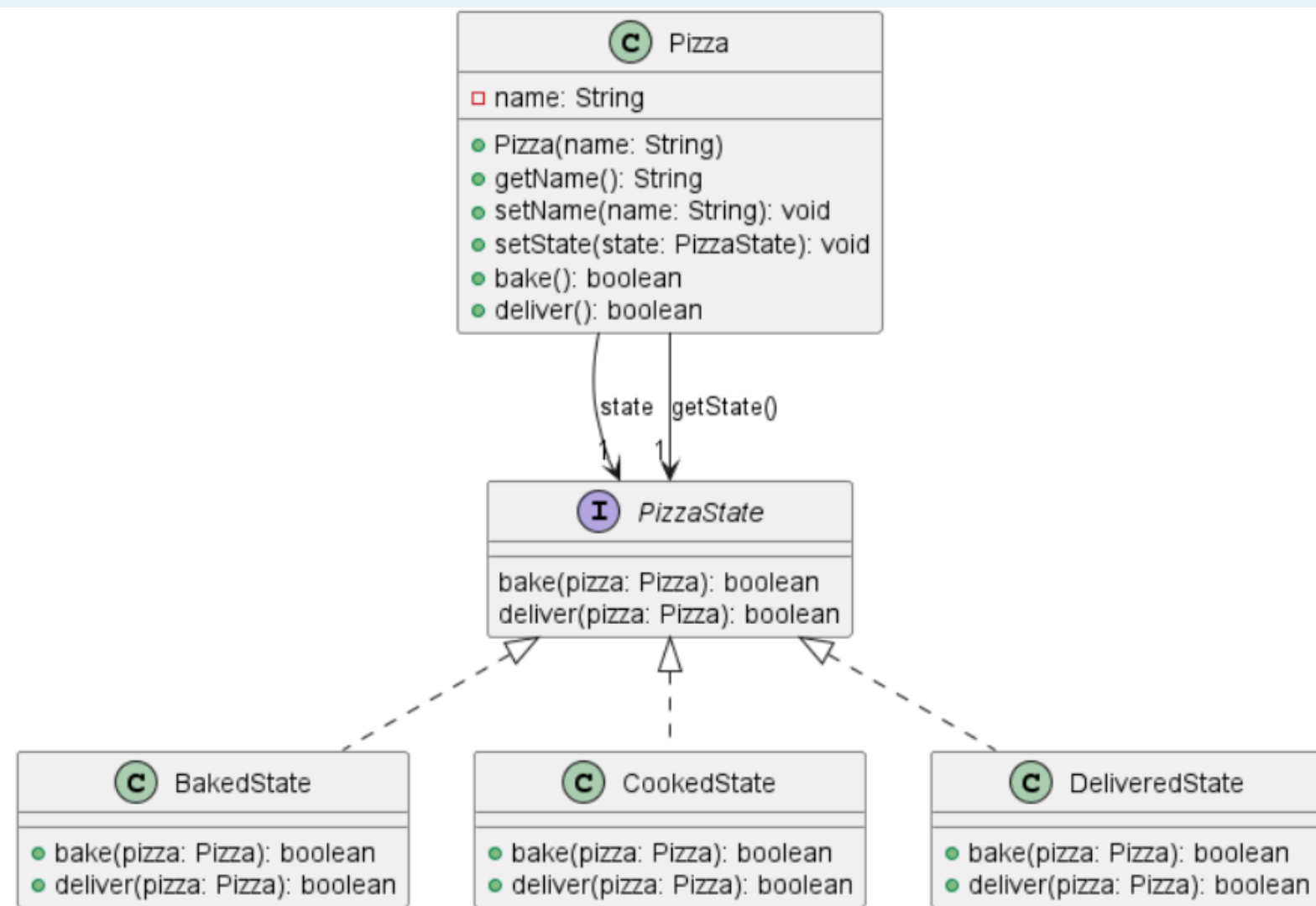
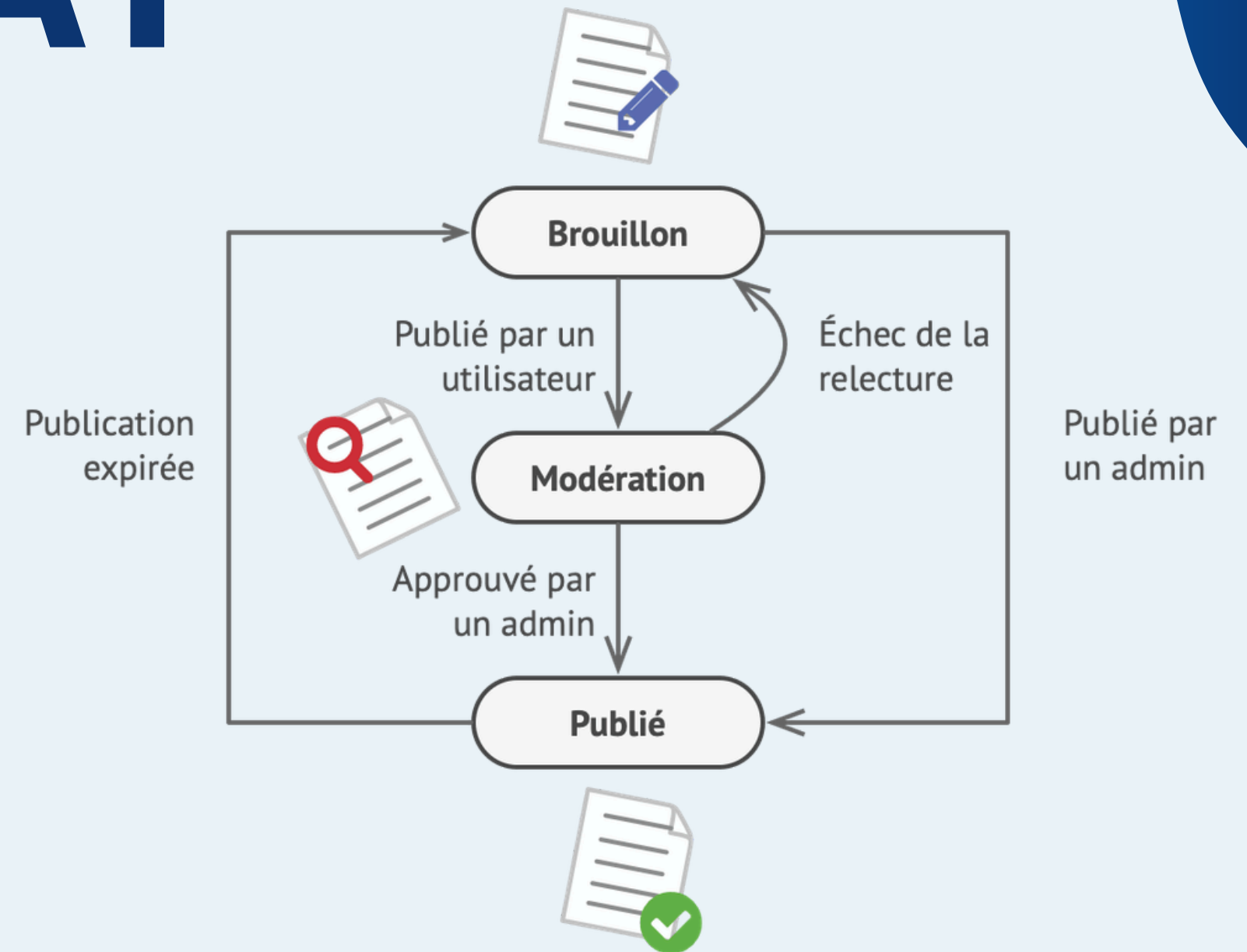
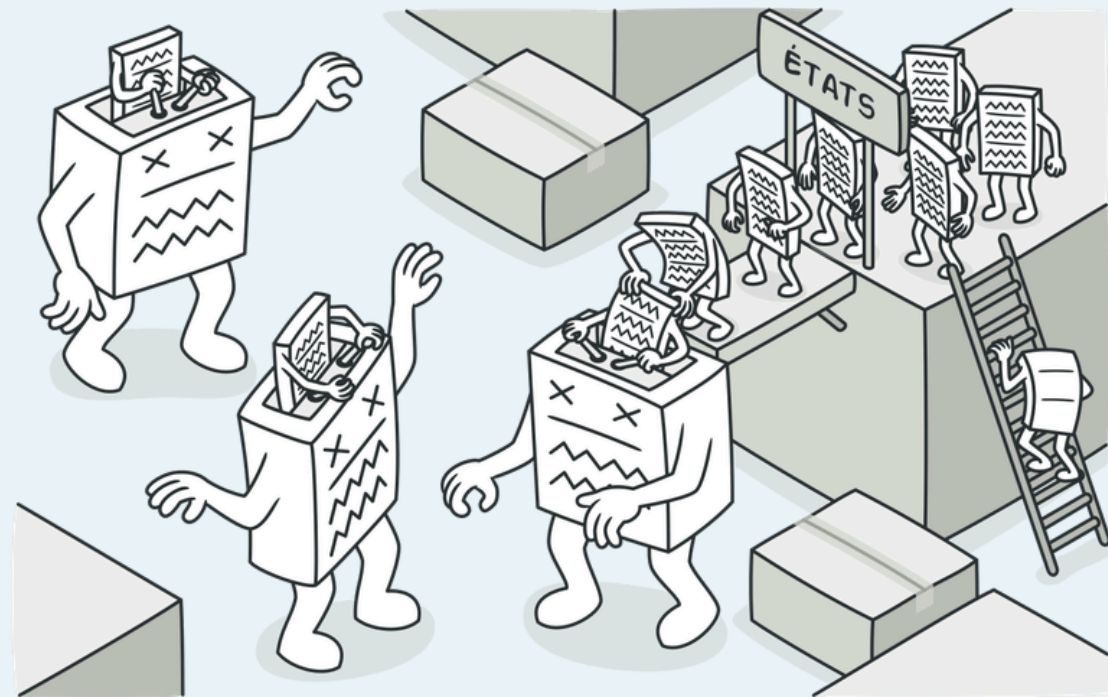


DIAGRAMME GÉNÉRIQUE



LE PATTERN ETAT

Pattern de conception comportemental



Comment modifier le comportement d'un objet lorsque son état interne change, au point de lui donner l'impression de changer de classe ?



SOLUTION PAR LA LITTÉRATURE

DIAGRAMME DE CLASSE

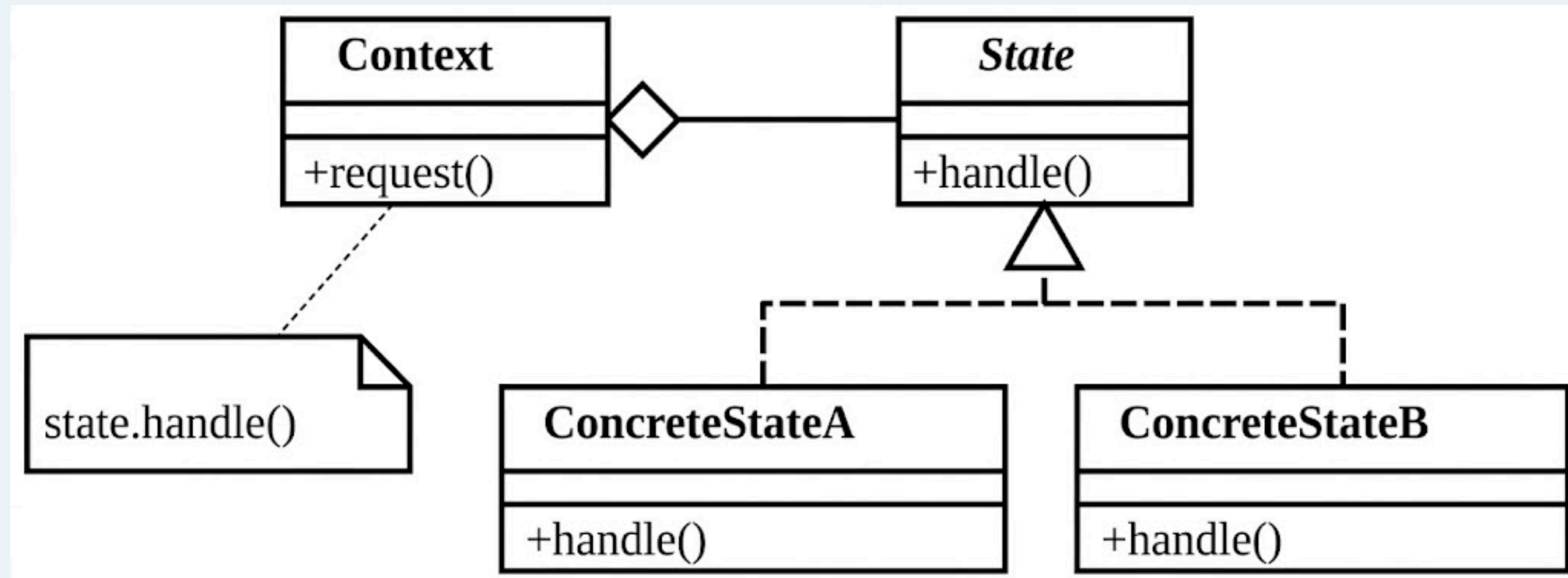
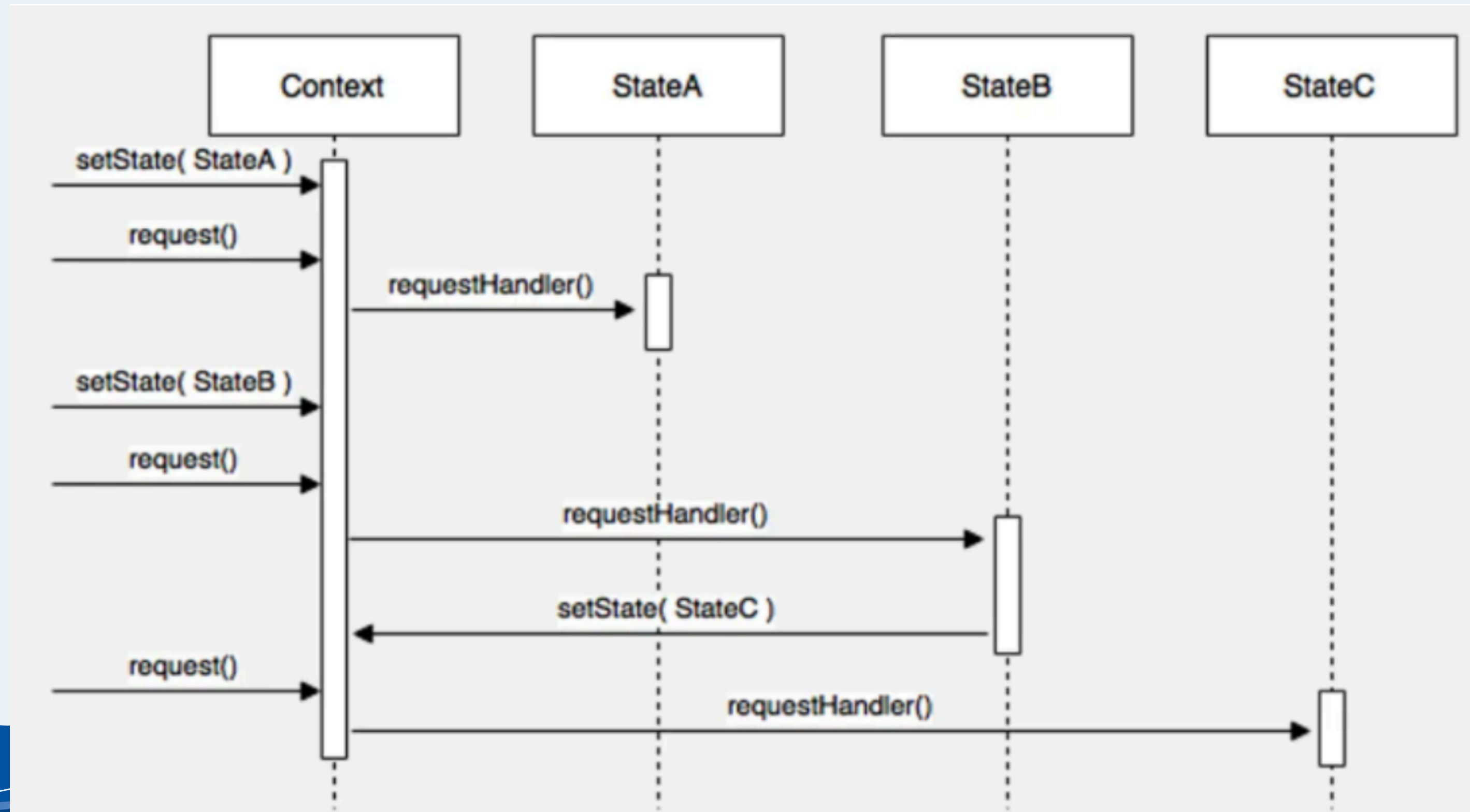


DIAGRAMME DE SÉQUENCE



ROLE DES CLASSES

Nom Générique	Description
Context	élément principal qui possède l'état. il peut être changé grâce au setter
State	Interface commune qui permet aux ConcreteState d'être instancié dans Context. Il possède les méthodes et sert de passerelle
ConcreteState	Classe qui implémente l'interface afin de donner accès aux méthodes nécessaires à la Classe dont elle est l'attribut.

LIEN AVEC SOLID

- SRP :** Extrait les comportements de la classe principale pour les déléguer à des classes spécifiques.
- OCP :** On ne change pas le code, on ajoute des états.
- LSP :** Les classes ConcreteState sont utilisées à la place de la classe State.

LIMITES DU PATTERN

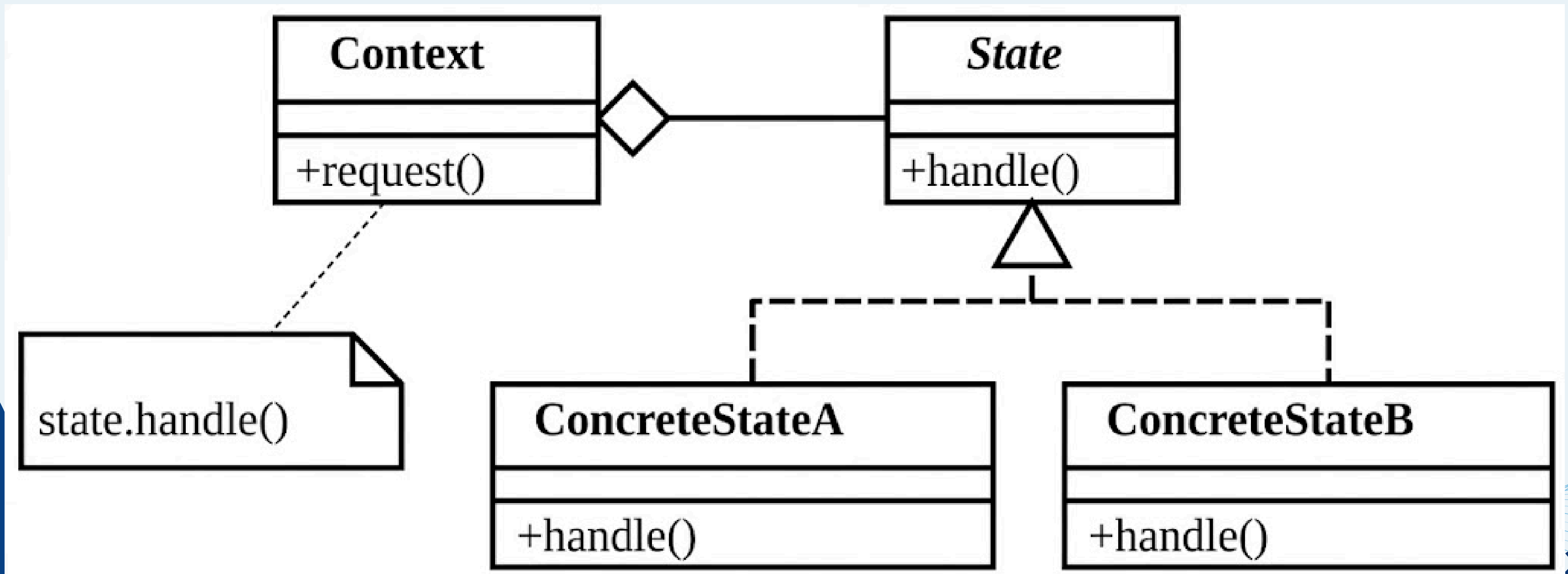
Excessif :

La mise en place du pattern demande de la réflexion en termes de conception et de codage. Il peut être excessif de l'utiliser s'il n'y a pas trop de changements d'état.

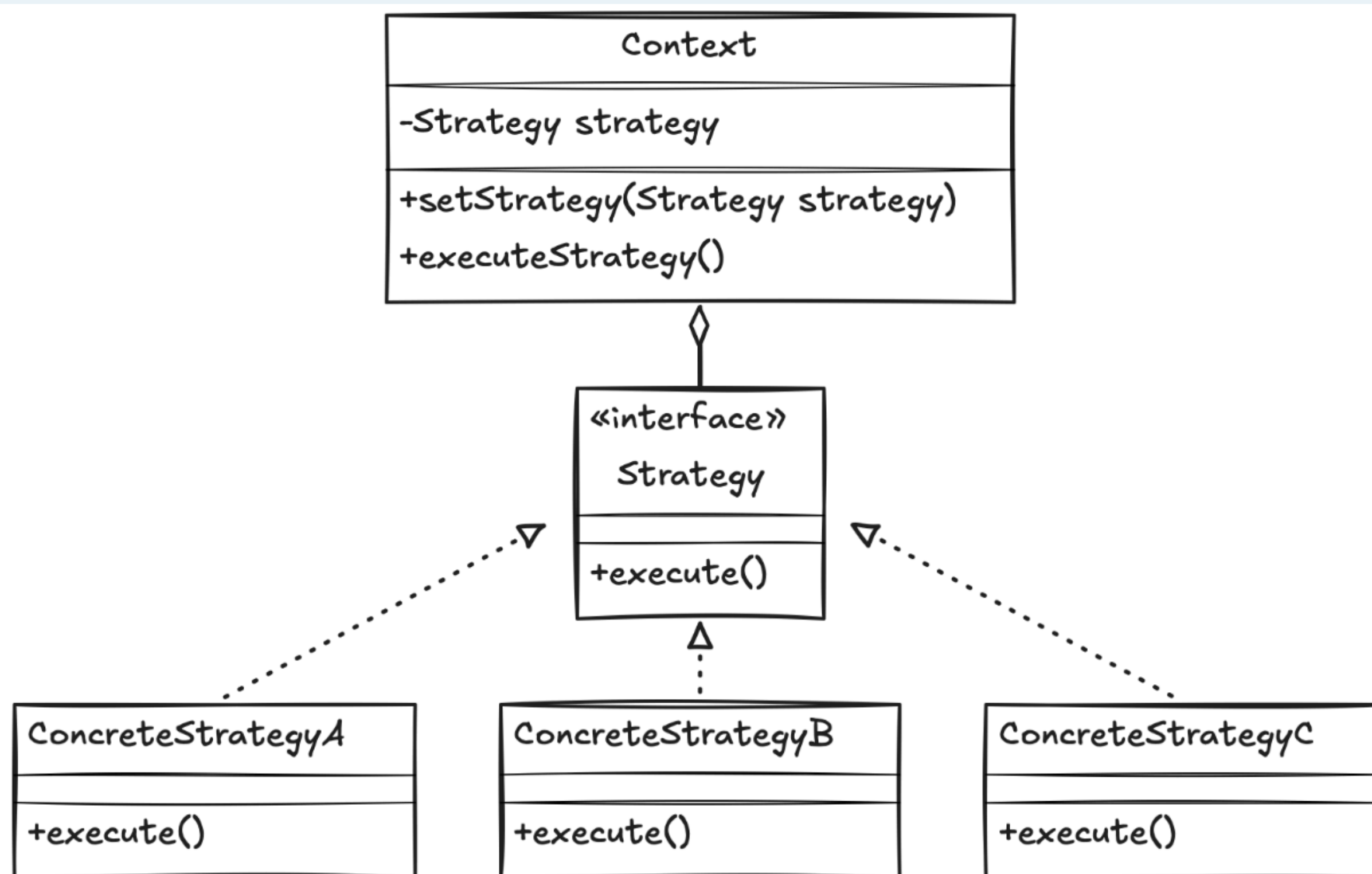
Multiplie le nombre de classes

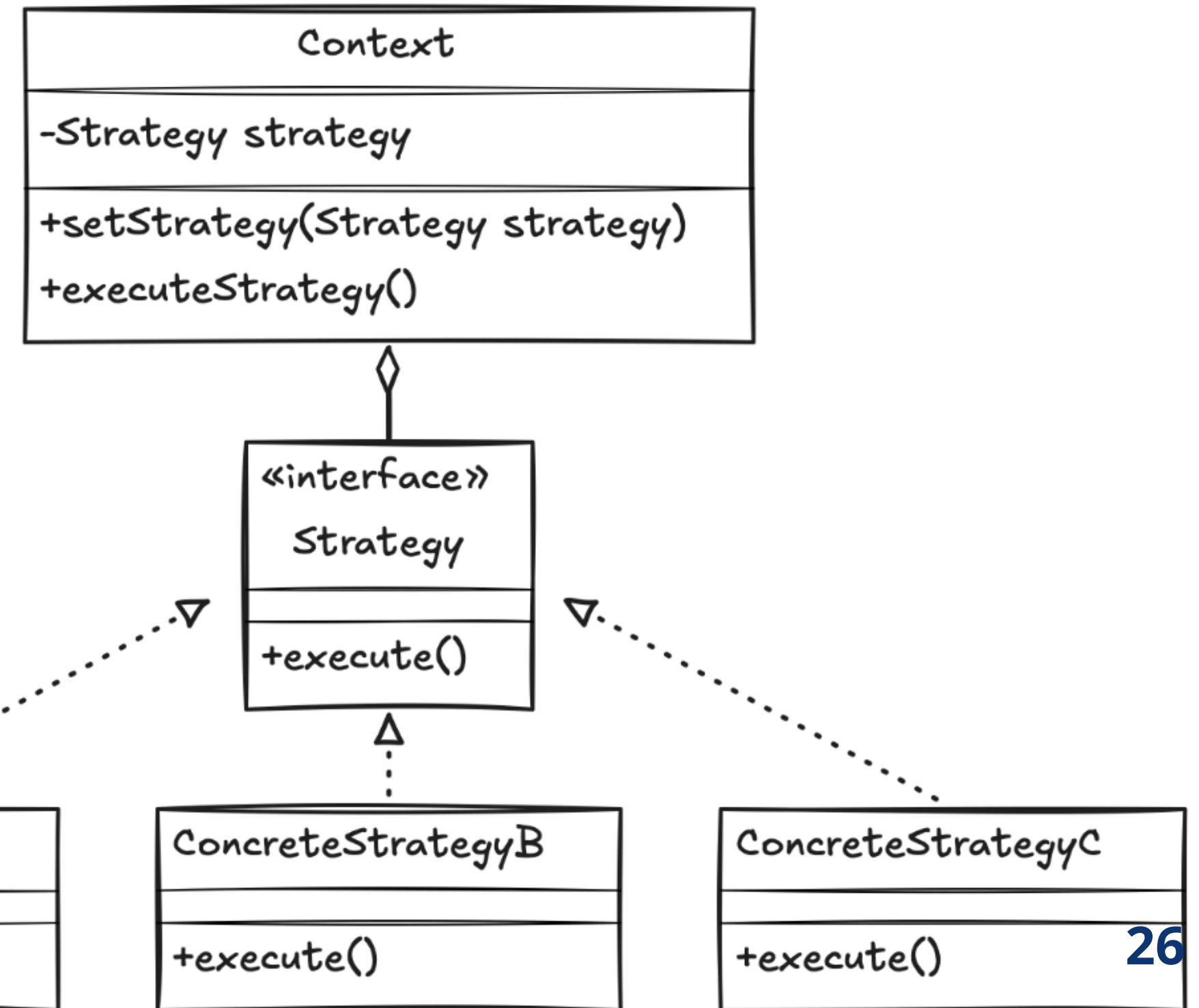
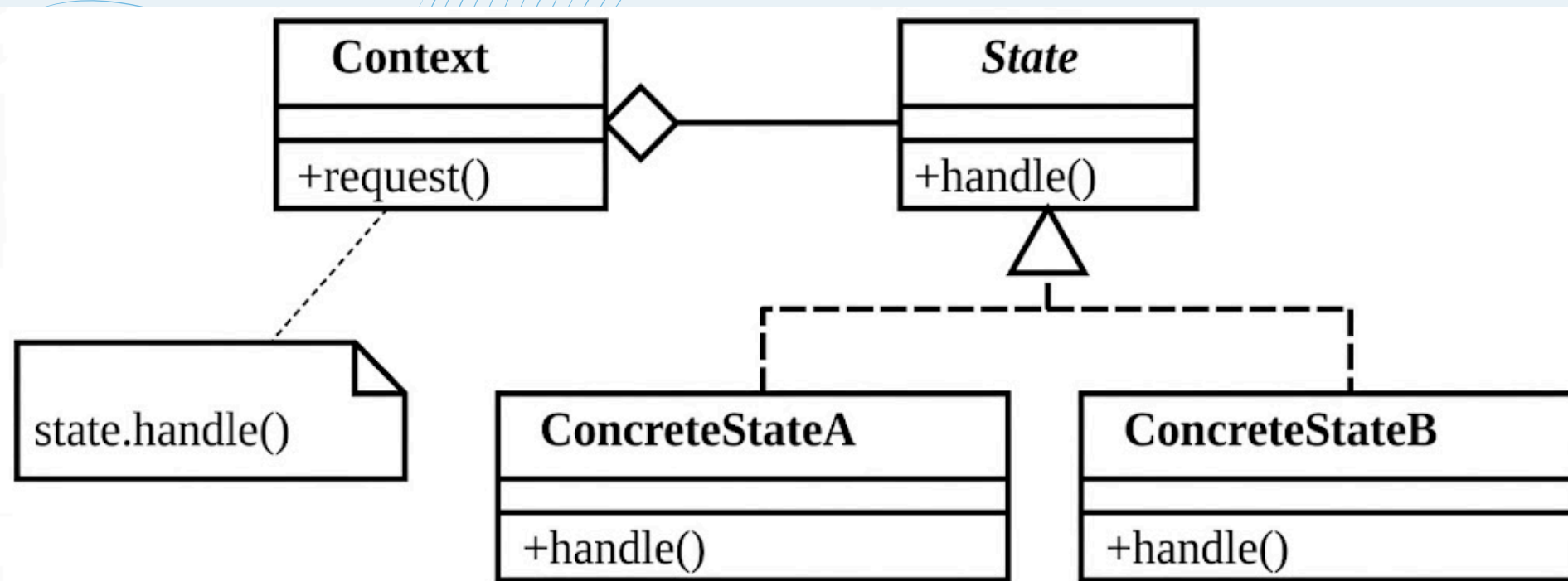
LIEN AVEC STRATEGIE

ETAT

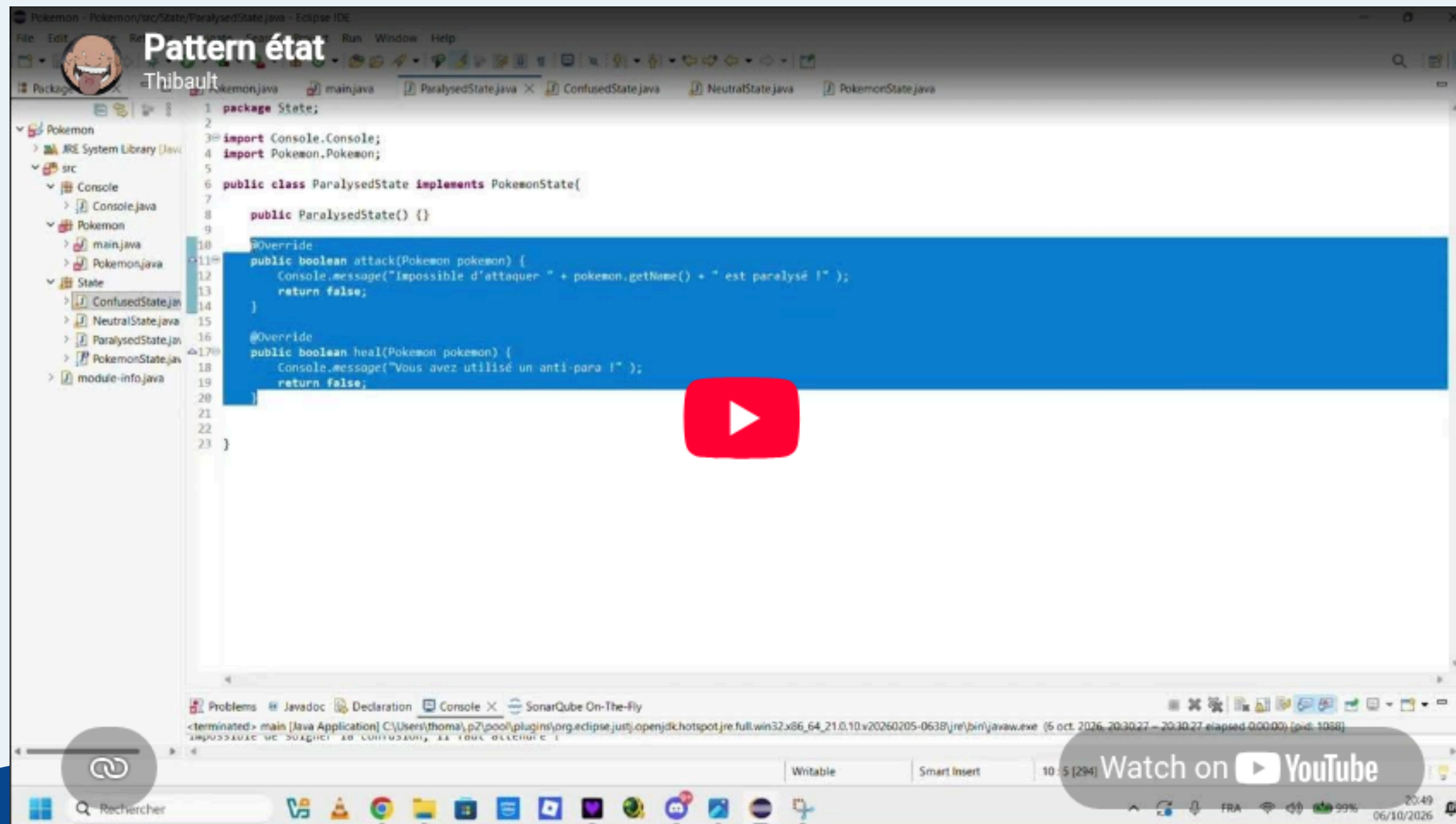


STRATÉGIE





LIVE CODING





QCM

SITOGRAPHIE

- <https://refactoring.guru/fr/design-patterns/state>
- http://goprod.bouhours.net/?page=pattern&pat_id=20
- https://sourcemaking.com/design_patterns/state
- <https://youtu.be/7N8fE4sV8js?si=xETM4qn9oy9qwKoY>
- https://en.wikipedia.org/wiki/Design_Patterns
- [https://fr.wikipedia.org/wiki/%C3%89tat_\(patron_de_conception\)](https://fr.wikipedia.org/wiki/%C3%89tat_(patron_de_conception))
- <https://gotan.io/fr/blog/design-patterns-gof/>

IAGRAPHIE

Création d'image absolument nécessaire

Modification d'image





MERCI